

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Monitoreo del estado de la batería con BroadcastReceiver en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	26/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS PROPUESTOS - Link Repositorio (Propuesta 1): https://github.com/shanccom/IDNP_Repo/tree/main/lab_04_Battery/BatteryMonitor_Compose - Link Repositorio (Propuesta 2): https://github.com/DeniseHuacani/IDNP-LABORATORIO/tree/main/lab04/BatteryMonitor_Compose Ejercicio Resuelto: Monitoreo de batería con BroadcastReceiver en Jetpack Compose Enunciado Implementar una pantalla en Jetpack Compose que muestre el nivel de batería en tiempo real mediante un BroadcastReceiver registrado dinámicamente con DisposableEffect. Desarrollo 1. Registro y Ciclo de Vida con DisposableEffect Se implementa el componente composable que escucha los cambios del sistema (ACTION_BATTERY_CHANGED) y calcula el porcentaje: <pre> 1 @Composable 2 fun BatteryScreen(modifier: Modifier = Modifier) { 3 var porcentaje by remember { mutableIntStateOf(0) } 4 val context = LocalContext.current 5 6 DisposableEffect(Unit) { 7 val receiver = object : BroadcastReceiver() { 8 override fun onReceive(context: Context?, intent: Intent?) { 9 val nivel = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1 10 val escala = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1 11 if (nivel != -1 && escala != -1) { 12 porcentaje = (nivel * 100) / escala 13 } 14 } 15 } 16 context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED)) 17 } 18 } </pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

14     }
15 }
16 context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
17
18 onDispose {
19     context.unregisterReceiver(receiver)
20 }
21 }
22
23 Text(text = "Batería: $porcentaje%", modifier = modifier)
24 }

```

2. Aspectos Clave

- **Extracción de datos:** Se obtienen EXTRA_LEVEL y EXTRA_SCALE para calcular el porcentaje de carga.
- **Gestión de recursos:** DisposableEffect registra el receiver al entrar en composición y lo desregistra en onDispose para evitar fugas de memoria.
- **Estado optimizado:** mutableIntStateOf(0) previene el autoboxing de tipos primitivos en recomposiciones frecuentes.

3. Evidencias de Ejecución y Pruebas

Se validó el comportamiento reactivo en tiempo real modificando el nivel de batería desde el emulador en los entornos de desarrollo de ambos integrantes del equipo:

Pruebas de Integrante 1

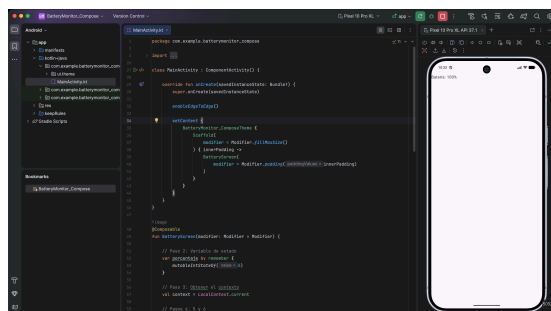


Figura 1: Ejecución inicial al 100%.

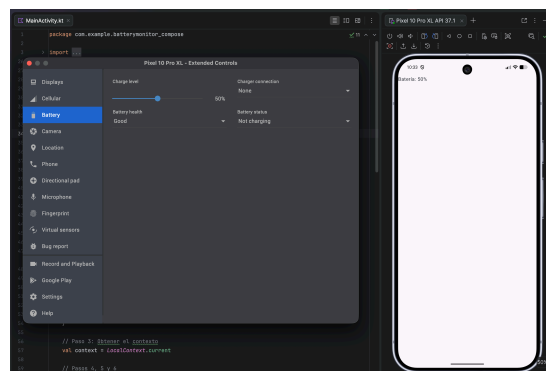


Figura 2: Simulación de nivel al 50%.

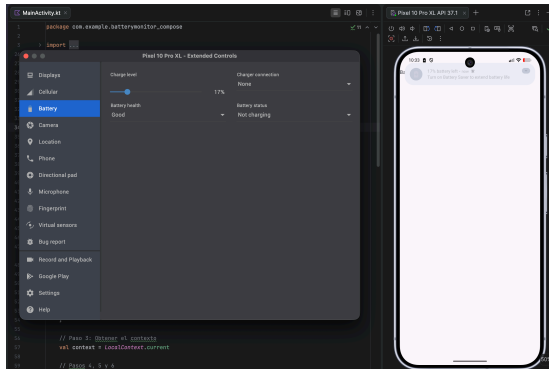


Figura 3: Simulación de nivel crítico al 17%.

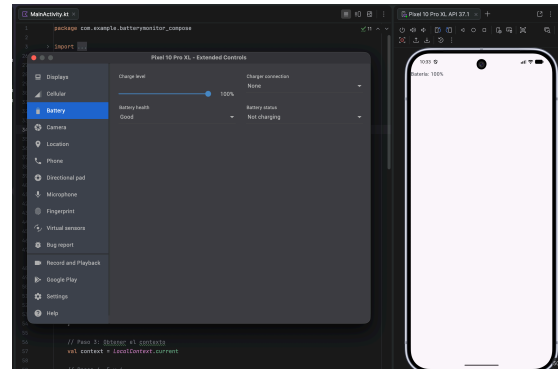


Figura 4: Restablecimiento de batería al 100%.

Pruebas de Integrante 2

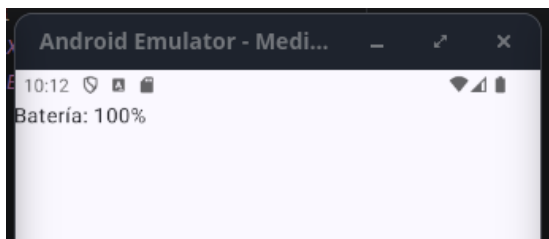


Figura 5: Ejecución inicial al 100%.

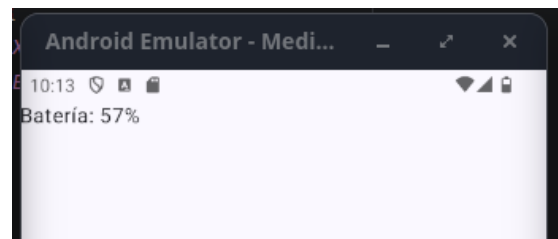


Figura 6: Simulación de nivel al 57%.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

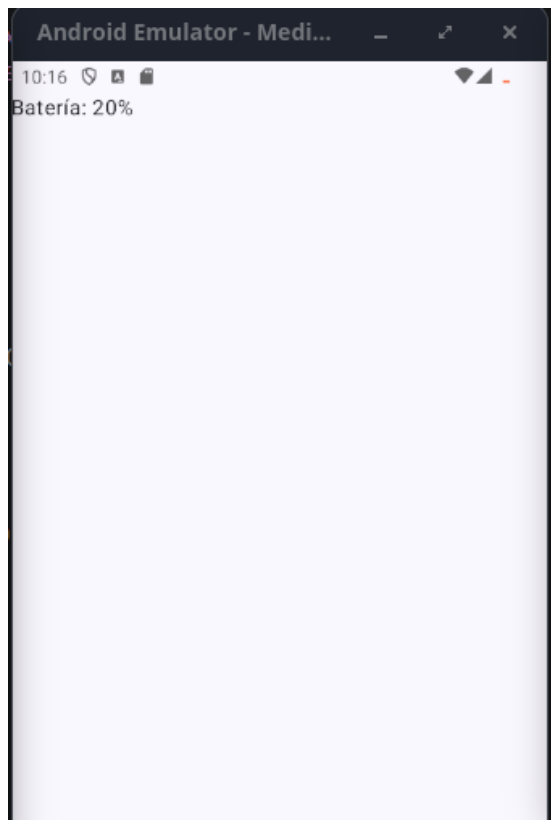


Figura 7: Simulación de nivel crítico al 20%.

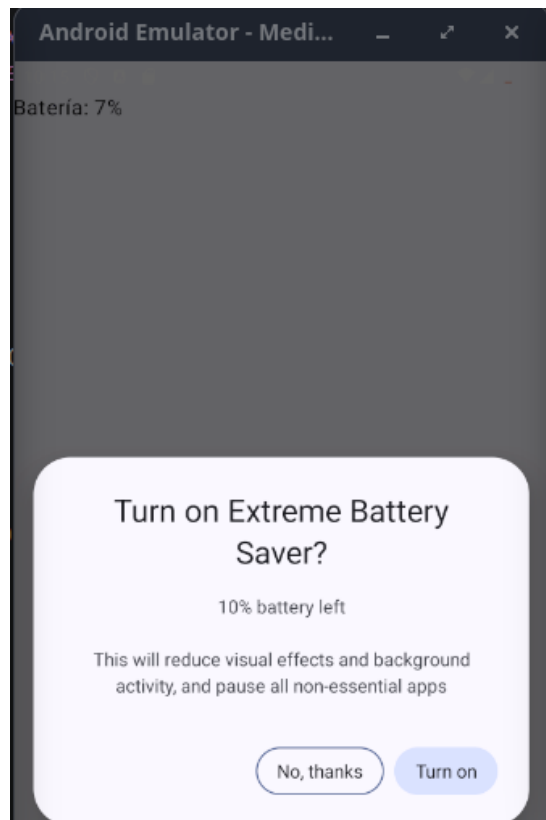


Figura 8: Simulación de nivel al 7%.

Modificaciones para Warnings y Buenas Prácticas

- Manejo de `innerPadding`: Se aplicó el `innerPadding` provisto por `Scaffold` para evitar superposiciones con barras del sistema y prevenir advertencias de layout en `Compose`.
- Parámetro `Modifier`: Se añadió `modifier: Modifier = Modifier` a `BatteryScreen()` para garantizar su reusabilidad y cumplir con las directrices oficiales de Android.
- Optimización con `mutableIntStateOf`: Se sustituyó `mutableStateOf(0)` por `mutableIntStateOf(0)` para optimizar el rendimiento y evitar el boxing innecesario de enteros.
- Validación y Lint: Se comprobó la compilación, pruebas y análisis estático con Android Lint, asegurando un código limpio, funcional y libre de errores.

PROPUESTA 1

Ejercicio Propuesto : Actualización manual del porcentaje de batería con broadcast personalizado

Enunciado

Tomando como referencia el ejercicio resuelto por el docente, agregar una segunda forma de actualizar el porcentaje de batería, disparada manualmente en lugar de por el evento automático del sistema:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

1. Agregar un botón «Actualizar manualmente» a BatteryScreen que cree un Intent con una acción propia (com.tuapp.ACTUALIZAR_BATERIA), lo envuelva en un PendingIntent.getBroadcast(...) y lo active con PendingIntent.send().
2. Implementar un segundo BroadcastReceiver que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería y actualice el mismo estado en pantalla.
3. Registrar y desregistrar este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.
4. (Reto opcional) Mostrar en pantalla, además del porcentaje, si la batería se está cargando o no, usando EXTRA_STATUS del Intent original.

Desarrollo y Solución Paso a Paso

Paso 1: Botón «Actualizar manualmente» y emisión con PendingIntent

Se define la acción propia y la función para encapsular el intent en un PendingIntent inmutable que dispara el broadcast bajo demanda al presionar el botón:

```

1  const val ACCION_ACTUALIZAR_BATERIA = "com.tuapp.ACTUALIZAR_BATERIA"
2
3  fun enviarActualizacionManual(context: Context) {
4      val intent = Intent(ACCION_ACTUALIZAR_BATERIA).apply {
5          setPackage(context.packageName) // Seguridad interna
6      }
7      val pendingIntent = PendingIntent.getBroadcast(
8          context,
9          0,
10         intent,
11         PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
12     )
13     pendingIntent.send()
14 }
```

Análisis: PendingIntent delega la ejecución al sistema; la bandera FLAG_IMMUTABLE impide que aplicaciones externas alteren el contenido del intent, mientras que setPackage() asegura que la transmisión ocurra exclusivamente dentro del paquete de la app.

Paso 2: Implementación del segundo BroadcastReceiver personalizado

Se implementa el segundo receptor encargado de capturar la acción manual y consultar sincronamente el estado del hardware:

```

1  val receiverManual = object : BroadcastReceiver() {
2      override fun onReceive(context: Context?, intent: Intent?) {
3          // Consulta instantánea del último broadcast sticky del sistema
4          val stickyIntent = context?.registerReceiver(null,
5              IntentFilter(Intent.ACTION_BATTERY_CHANGED))
6          val nivel = stickyIntent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1
7          val escala = stickyIntent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1
8          if (nivel != -1 && escala != -1) {
9              porcentaje = (nivel * 100) / escala
10         }
11     }
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

Análisis: Al pasar null como receptor en registerReceiver(), Android retorna de inmediato el último intent pegajoso (sticky broadcast) sin crear una suscripción activa, permitiendo una lectura puntual y eficiente bajo demanda.

Paso 3: Registro y desregistro del segundo receptor con DisposableEffect

Se integra el segundo receiver en la jerarquía de Compose respetando su ciclo de vida:

```

1 DisposableEffect(Unit) {
2     val filtroManual = IntentFilter(ACCION_ACTUALIZAR_BATERIA)
3     ContextCompat.registerReceiver(
4         context,
5         receiverManual,
6         filtroManual,
7         ContextCompat.RECEIVER_NOT_EXPORTED
8     )
9
10    onDispose {
11        context.unregisterReceiver(receiverManual)
12    }
13 }

```

Análisis: Acoplar la suscripción a DisposableEffect garantiza que el receptor se desregistre automáticamente al desmontarse la vista (onDispose), evitando fugas de memoria (memory leaks) en la aplicación.

Paso 4: (Reto opcional) Detección y visualización del estado de carga (EXTRA_STATUS)

Se extrae EXTRA_STATUS para evaluar si el dispositivo recibe energía eléctrica y reflejarlo en un indicador visual:

```

1 val status = intent?.getIntExtra(BatteryManager.EXTRA_STATUS, -1) ?: -1
2 val estaCargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
3     status == BatteryManager.BATTERY_STATUS_FULL

```

Análisis: Es indispensable validar tanto BATTERY_STATUS_CHARGING como BATTERY_STATUS_FULL, ya que un dispositivo al 100% continúa enchufado a la corriente aunque el proceso de carga activa haya finalizado.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

5. Evidencias de Ejecución y Pruebas

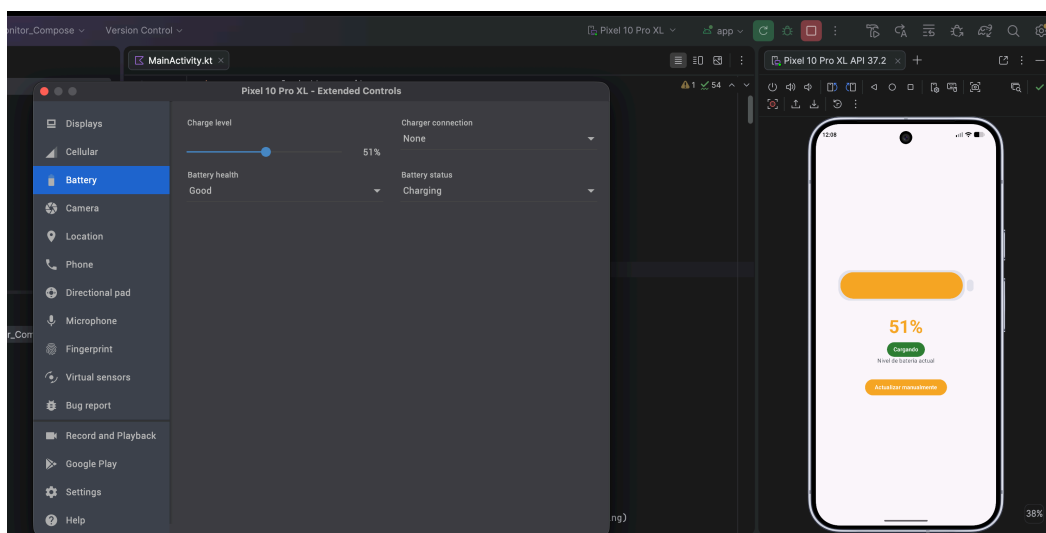


Figura 9: Simulación en Extended Controls al 51% en estado «Charging» reflejado con píldora verde «Cargando».

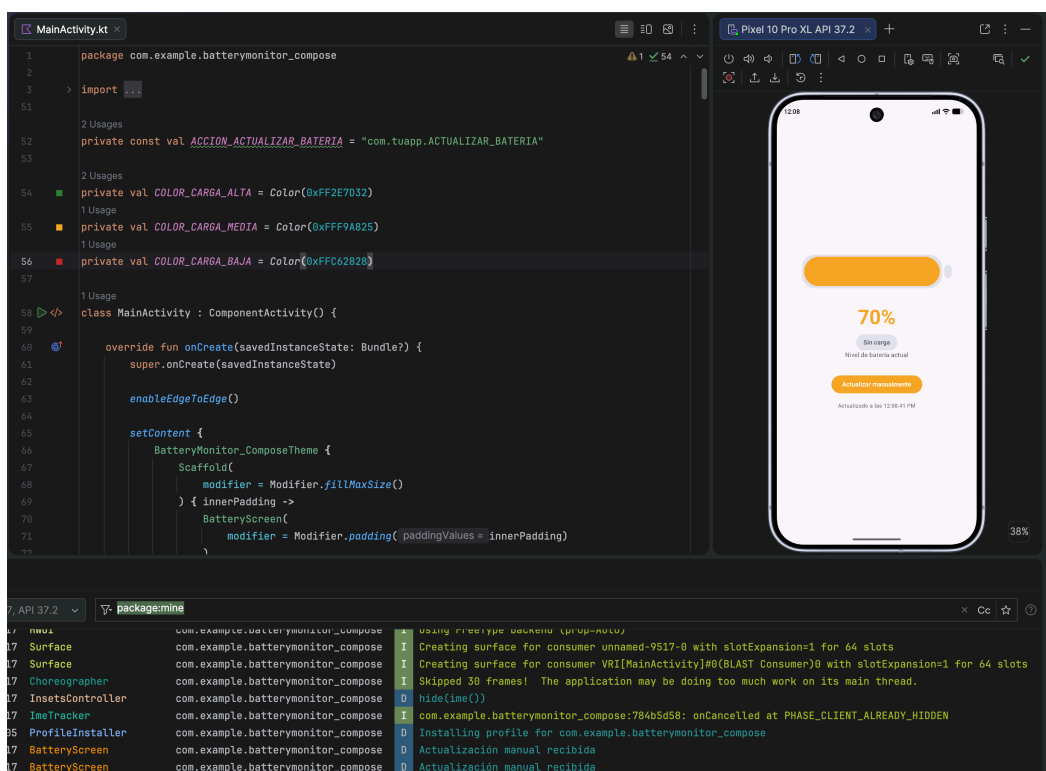


Figura 10: Actualización manual al 70% («Sin carga») y verificación del broadcast en Logcat («Actualización manual recibida»).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

PROPUESTA 2

Ejercicio Propuesto: Actualización manual del porcentaje de batería con broadcast personalizado

Enunciado

Tomando como referencia el ejercicio resuelto por el docente, agregar una segunda forma de actualizar el porcentaje de batería, disparada manualmente en lugar de por el evento automático del sistema:

1. Agregar un botón «Actualizar manualmente» a BatteryScreen que cree un Intent con una acción propia (com.tuapp.ACTUALIZAR_BATERIA), lo envuelva en un PendingIntent.getBroadcast(...) y lo active con PendingIntent.send().
2. Implementar un segundo BroadcastReceiver que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería y actualice el mismo estado en pantalla.
3. Registrar y desregistrar este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.
4. **(Reto opcional)** Mostrar en pantalla, además del porcentaje, si la batería se está cargando o no, usando EXTRA_STATUS del Intent original.

Desarrollo y Solución Paso a Paso

Paso 1: Agrega un botón «Actualizar manualmente» a BatteryScreen. Al presionarlo, en vez de esperar el evento del sistema, dispara un broadcast personalizado: crea un Intent con una acción propia (por ejemplo «com.tuapp.ACTUALIZAR_BATERIA»), envuélvelo en un PendingIntent.getBroadcast(...), y actívalo con PendingIntent.send().

En este ejercicio se agrega a BatteryScreen un botón que permite forzar la actualización del porcentaje de batería sin depender del evento automático ACTION_BATTERY_CHANGED. Para lograrlo se define una acción personalizada, se envuelve en un PendingIntent y se envía como un broadcast propio al presionar el botón.

1. Definir la acción personalizada

Se declara, fuera del Composable, una constante con el nombre de la acción propia que identificará a este broadcast dentro de la aplicación:

```
1  const val ACTION_ACTUALIZAR_BATERIA = "com.tuapp.ACTUALIZAR_BATERIA"
```

2. Nuevos imports

Se agregan PendingIntent (para envolver el Intent) y los componentes de UI necesarios para el botón:

```
1  import android.app.PendingIntent
2  import androidx.compose.foundation.layout.Column
3  import androidx.compose.material3.Button
```

3. Agregar el botón «Actualizar manualmente»

Dentro del cuerpo de BatteryScreen, se agrega un Button. Al presionarlo se construye un Intent con la acción personalizada, se envuelve en un PendingIntent.getBroadcast(...) y se dispara con .send():

```
1  Column(modifier = modifier) {
2      Text("Batería: $porcentaje%")
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```

3
4      Button(onClick = {
5          val intent = Intent(ACTION_ACTUALIZAR_BATERIA).apply {
6              setPackage(context.packageName)
7          }
8
9          val pendingIntent = PendingIntent.getBroadcast(
10             context,
11             0,
12             intent,
13             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
14         )
15
16         pendingIntent.send()
17     }) {
18         Text("Actualizar batería manualmente")
19     }
20 }

```

4. Verificación

Al ejecutar el proyecto se muestra el botón «Actualizar manualmente» debajo del texto de porcentaje. Presionarlo no produce aún ningún cambio visible en pantalla (porque todavía no existe el receiver que escuche esta acción), pero en Logcat puede confirmarse que el broadcast se envía sin errores:

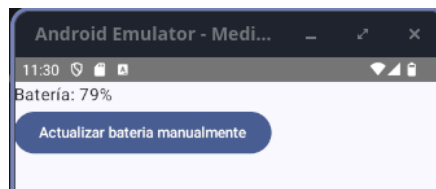


Figura 11: Botón «Actualizar manualmente» agregado a BatteryScreen

Paso 2: Implementa un segundo BroadcastReceiver (o reutiliza el mismo patrón) que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería (por ejemplo con BatteryManager) y actualice el mismo estado en pantalla.

En este ejercicio se implementa el receptor encargado de procesar la acción personalizada disparada en el Ejercicio 1. Cuando se presiona el botón «Actualizar manualmente», el PendingIntent emite la acción com.tuapp.ACTUALIZAR_BATERIA. Este BroadcastReceiver la intercepta, consulta el estado actual de la energía y actualiza la variable porcentaje en la interfaz.

1. El problema: el Intent personalizado no trae los extras de batería

A diferencia del evento del sistema ACTION_BATTERY_CHANGED, el Intent de nuestra acción personalizada com.tuapp.ACTUALIZAR_BATERIA llega completamente «vacío» (sin datos extras de carga).

Por ello, dentro de onReceive(), no se leen los extras del intent recibido. En su lugar, se consulta al sistema operativo por el último estado conocido utilizando un registro temporal con null: al pasar null como receptor

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

en registerReceiver(), Android nos devuelve directamente el último Intent «sticky» de la batería sin dejar un receptor registrado de forma permanente.

2. Crear el segundo BroadcastReceiver

Se define el objeto receiverManual. Al recibir la señal, realiza la lectura del estado mediante el receptor temporal, extrae los valores crudos, calcula el porcentaje real y refresca la pantalla:

```

1  val receiverManual = object : BroadcastReceiver() {
2      override fun onReceive(context: Context?, intent: Intent?) {
3          // 1. Obtener la captura más reciente de la batería (Intent sticky)
4          val batteryStatus = context?.registerReceiver(
5              null,
6              IntentFilter(Intent.ACTION_BATTERY_CHANGED)
7          )
8
9          // 2. Extraer el nivel actual y la escala máxima de carga
10         val nivel = batteryStatus?.getIntExtra(BatteryManager.EXTRA_LEVEL,
11             -1) ?: -1
12
13         val escala = batteryStatus?.getIntExtra(BatteryManager.EXTRA_SCALE,
14             -1) ?: -1
15
16         // 3. Calcular el porcentaje correspondiente de 0 a 100%
17         if (nivel != -1 && escala != -1) {
18             porcentaje = (nivel * 100) / escala
19         }
20
21         Log.d("BatteryScreen", "Actualización manual recibida")
22     }
23 }

```

```

val receiverManual = remember {
    object : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val batteryStatus = context?.registerReceiver(
                receiver = null,
                filter = IntentFilter( action = Intent.ACTION_BATTERY_CHANGED)
            )

            val nivel = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_LEVEL, defaultValue = -1) ?: -1
            val escala = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_SCALE, defaultValue = -1) ?: -1

            if (nivel != -1 && escala != -1) {
                porcentaje = (nivel * 100) / escala
            }

            Log.d( tag = "BatteryScreen", msg = "Actualización manual recibida")
        }
    }
}

```

Figura 12: Segundo BroadcastReceiver implementado

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

Paso 3: Registra y desregistra este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.

En este ejercicio se activa el `receiverManual` definido en el Ejercicio 2, registrándolo con el sistema dentro de un `DisposableEffect` independiente, y liberándolo correctamente cuando el Composable sale de pantalla, siguiendo el mismo patrón que usa el docente para el receiver automático.

1. Segundo DisposableEffect

Se agrega un nuevo bloque `DisposableEffect(Unit)`, separado del que registra el receiver automático, dedicado exclusivamente al `receiverManual`. Dentro se llama a `context.registerReceiver(...)` filtrando por la acción personalizada y se imprime un mensaje de confirmación por consola:

```

1      DisposableEffect(Unit) {
2
3          ContextCompat.registerReceiver(
4              context,
5              receiverManual,
6              IntentFilter(ACTION_ACTUALIZAR_BATERIA),
7              ContextCompat.RECEIVER_NOT_EXPORTED
8          )
9          Log.d("BatteryScreen", "Receiver manual registrado")
10
11         onDispose {
12             context.unregisterReceiver(receiverManual)
13             Log.d("BatteryScreen", "Receiver manual desregistrado")
14         }
15     }

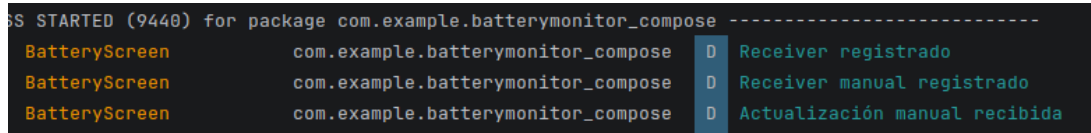
```

2. Por qué se usa RECEIVER_NOT_EXPORTED

A partir de Android 13 (API 33), todo `registerReceiver` dinámico debe declarar explícitamente si el receiver puede recibir broadcasts de otras aplicaciones (`RECEIVER_EXPORTED`) o solo de la propia app (`RECEIVER_NOT_EXPORTED`). Se usa `RECEIVER_NOT_EXPORTED` por seguridad, evitando que otras apps puedan activar este receiver.

3. Verificación

En Logcat, filtrando por la etiqueta «BatteryScreen», se observan los mensajes «Receiver manual registrado» al entrar a la pantalla, «Actualización manual recibida» cada vez que se presiona el botón, y «Receiver manual desregistrado» al salir:



```

SS STARTED (9440) for package com.example.batterymonitor_compose -----
BatteryScreen      com.example.batterymonitor_compose  D  Receiver registrado
BatteryScreen      com.example.batterymonitor_compose  D  Receiver manual registrado
BatteryScreen      com.example.batterymonitor_compose  D  Actualización manual recibida

```

Figura 13: Logcat mostrando registro, actualización manual y desregistro del segundo receiver

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

Paso 4 (Reto opcional): Muestra en pantalla, además del porcentaje, si la batería se está cargando o no, usando **EXTRA_STATUS** del Intent original.

En este ejercicio se agrega una segunda variable de estado que indica si el dispositivo está cargando, obtenida a partir de **EXTRA_STATUS**. Esta variable se actualiza tanto desde el receiver automático como desde el receiver manual, y se muestra en un segundo Text junto al porcentaje.

1. Nueva variable de estado

Se declara cargando, junto a porcentaje, para almacenar si la batería está en proceso de carga:

```
1 var cargando by remember { mutableStateOf(false) }
```

2. Leer EXTRA_STATUS en el receiver automático

Dentro de **onReceive()** del receiver del Ejercicio resuelto por el docente, además de **EXTRA_LEVEL** y **EXTRA_SCALE**, se extrae **EXTRA_STATUS** y se compara contra las constantes **BatteryManager.BATTERY_STATUS_CHARGING** y **BATTERY_STATUS_FULL** (batería llena conectada al cargador también cuenta como «cargando» a efectos prácticos):

```
1 val status = intent?.getIntExtra(BatteryManager.EXTRA_STATUS, -1) ?: -1
2
3 cargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
4           status == BatteryManager.BATTERY_STATUS_FULL
```

3. Leer EXTRA_STATUS en el receiver manual

El mismo cálculo se agrega dentro de **receiverManual**, ya que este también consulta el estado real de la batería mediante **registerReceiver(null, ...)** y por lo tanto también tiene acceso a **EXTRA_STATUS**:

```
1 val status = batteryStatus?.getIntExtra(BatteryManager.EXTRA_STATUS, -1) ?: -1
2
3 cargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
4           status == BatteryManager.BATTERY_STATUS_FULL
```

4. Mostrar el estado en pantalla

En el cuerpo de **BatteryScreen**, debajo del Text del porcentaje, se agrega un segundo Text que refleja cargando:

```
1 Text(text = if (cargando) "Estado: Cargando" else "Estado: No cargando")
```

5. Verificación

Desde **Extended Controls > Battery** del emulador, al cambiar el «Charge state» entre **Charging** y **Discharging**, el texto de estado cambia automáticamente entre «Estado: Cargando» y «Estado: No cargando», tanto por el evento del sistema como al presionar «Actualizar manualmente»:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

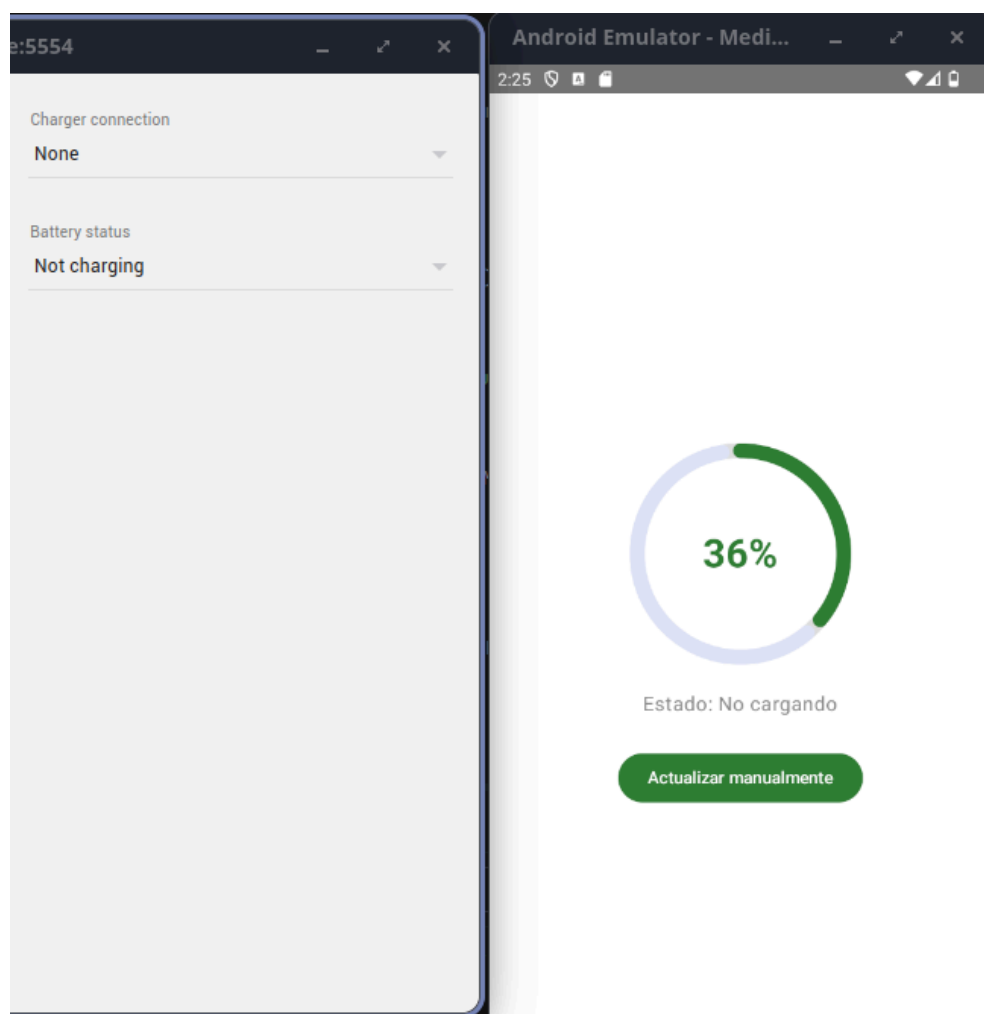


Figura 14: Pantalla mostrando porcentaje y estado de carga «NO CARGANDO»

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

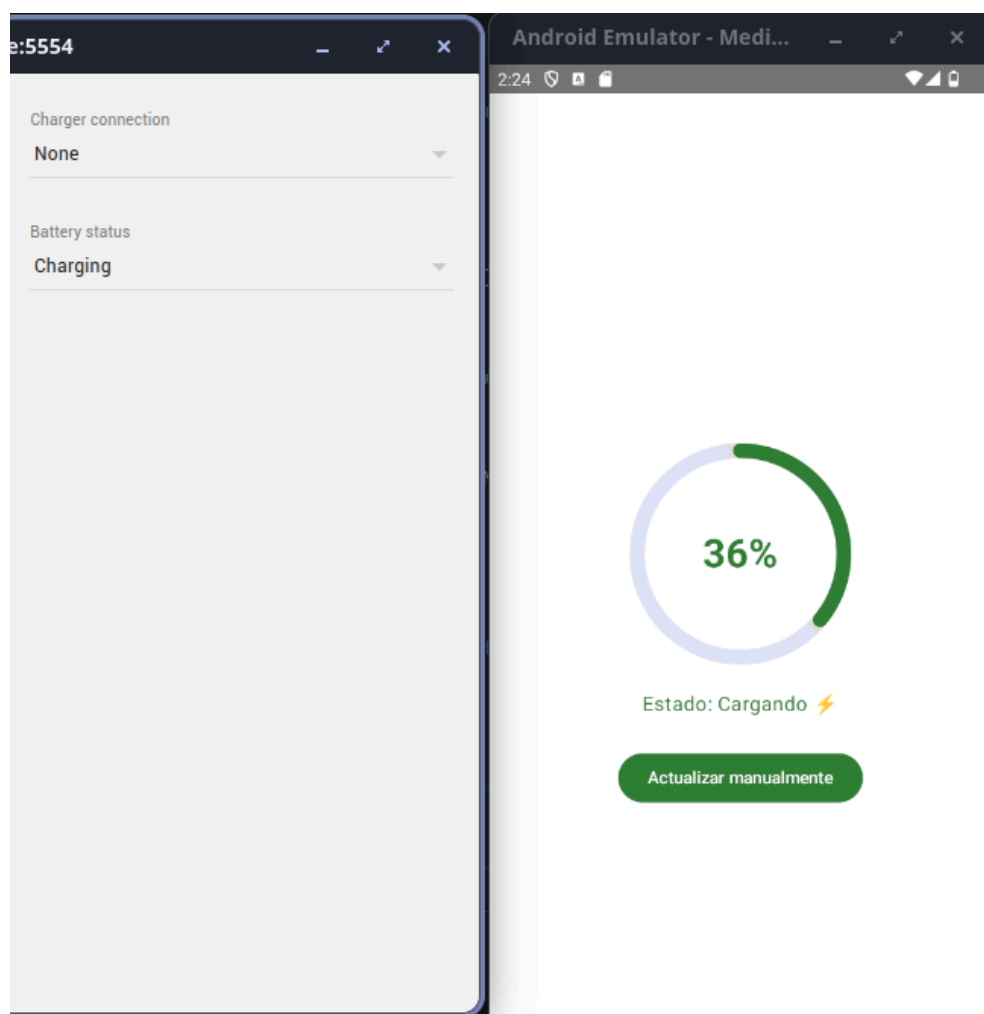


Figura 15: Pantalla mostrando porcentaje y estado de carga «CARGANDO»

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantalla que muestra el porcentaje de batería en tiempo real)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Respuesta

Sí, logramos resolver el ejercicio resuelto en su totalidad. Ambos integrantes implementamos correctamente el composable reactivo para escuchar los eventos del sistema. Confirmamos que el sistema actualiza inmediatamente el porcentaje de carga ante emisiones del hardware sin reiniciar la interfaz.

Evidencia de Ejecución — Integrante 1

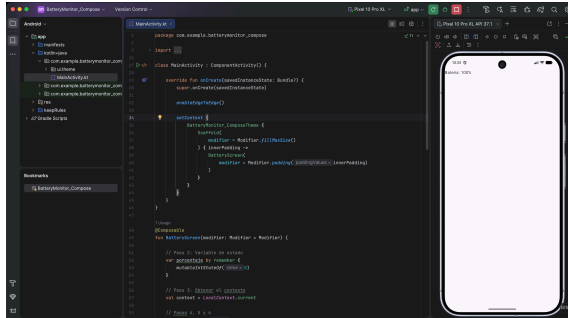


Figura 16: Ejecución inicial al 100%.

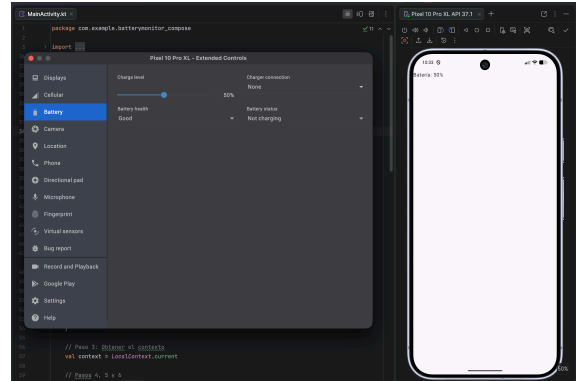


Figura 17: Simulación de respuesta al 50%.

Evidencia de Ejecución — Integrante 2

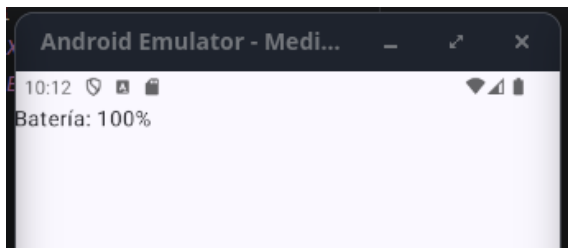


Figura 18: Ejecución inicial al 100%.

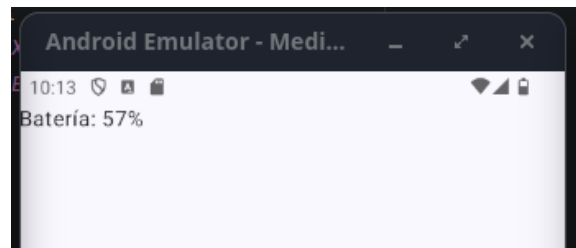


Figura 19: Simulación de nivel al 57%.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

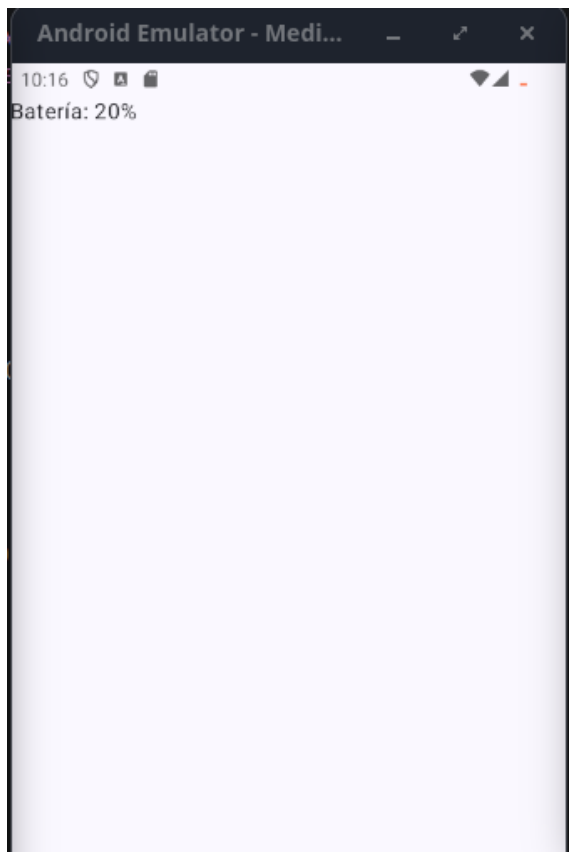


Figura 20: Simulación de nivel crítico al 20%.

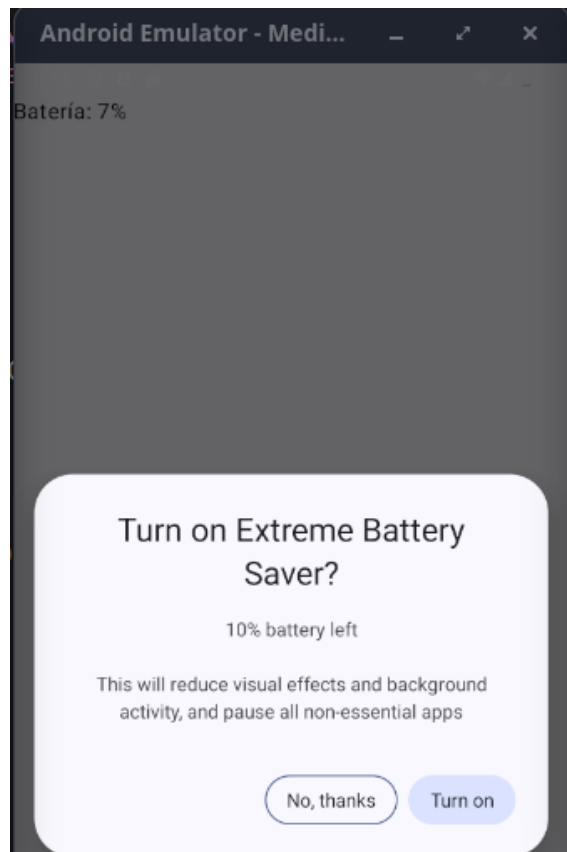


Figura 21: Simulación de nivel bajo al 7%.

Pregunta 2: En tu propia implementación, ¿qué diferencia encontraste entre recibir el evento automático del sistema (`ACTION_BATTERY_CHANGED`) y disparar un broadcast manualmente con `PendingIntent`? ¿En qué situaciones usarías cada uno?

Respuesta

La principal diferencia radica en el control del flujo: el sistema emite `ACTION_BATTERY_CHANGED` de manera pasiva y automática, mientras que con `PendingIntent` decido el momento exacto en que se dispara la actualización bajo demanda.

En la práctica, el evento del sistema es sticky y entrega el estado actual de inmediato al suscribirse; en cambio, el broadcast manual requiere disparar explícitamente el intent para gatillar la lectura.

Utilizaría `ACTION_BATTERY_CHANGED` para paneles de monitoreo continuo o ahorro de energía, y el broadcast manual con `PendingIntent` para botones interactivos o refrescos solicitados directamente por el usuario.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

Pregunta 3: En forma individual, comenta en el video de referencia Broadcast Receivers en Android con Jetpack Compose (YouTube) qué otras acciones del sistema, además del cambio de batería, se pueden escuchar con un BroadcastReceiver. (Colocar la captura del comentario; uno por integrante del grupo).

Respuesta

Realizamos nuestros comentarios individuales en el video de referencia, identificando eventos clave del sistema operativo que pueden ser monitoreados de forma reactiva con un BroadcastReceiver.

A continuación, se presentan las evidencias de la participación de ambos integrantes en YouTube:

Integrante 1

Realicé mi comentario destacando acciones esenciales del sistema como la transición al modo avión (ACTION_AIRPLANE_MODE_CHANGED), la conexión/desconexión a la corriente (ACTION_POWER_CONNECTED) y el cambio de conectividad de red.

Estas emisiones protegidas del sistema permiten que la aplicación reaccione inteligentemente al entorno del dispositivo sin necesidad de realizar sondeos constantes (polling), optimizando el uso de recursos.

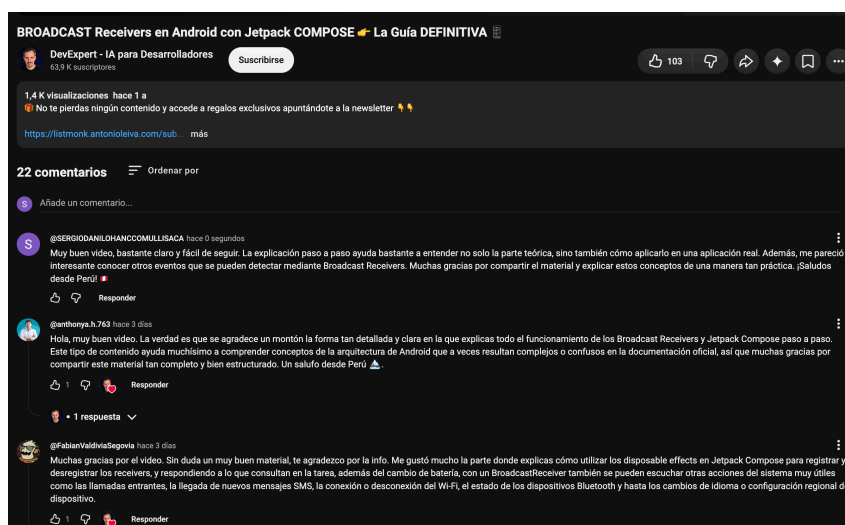


Figura 22: Captura del comentario realizado en el video de referencia de YouTube.

Integrante 2

Comente sobre el uso de DisposableEffect para prevenir fugas de memoria y también resalte el monitoreo de eventos como llamadas entrantes, mensajes SMS, estados de red Wi-Fi/Bluetooth y cambios de idioma con ACTION_LOCALE_CHANGED.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

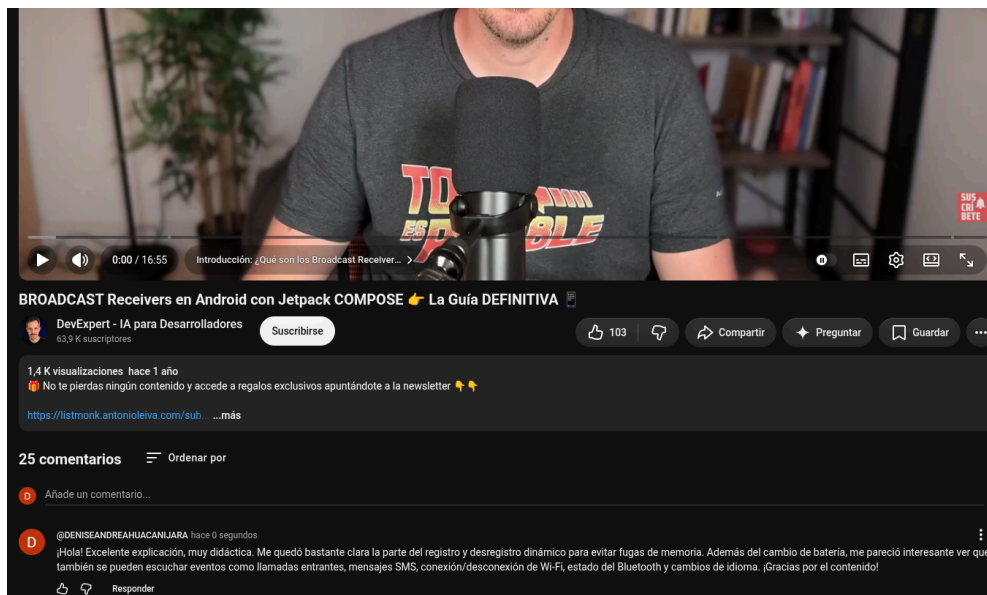


Figura 23: Comentario realizado por el Integrante 2 en YouTube.

III. CONCLUSIONES

- El laboratorio puso en evidencia un problema conceptual: la tensión entre el modelo de eventos de Android, imperativo y externo, y el modelo declarativo de Compose. Leer el `BroadcastReceiver` como un flujo unidireccional evento, estado, interfaz resultó el aprendizaje más transferible de la experiencia.
- El estudio de los sticky broadcasts reveló que el framework resuelve, por sí mismo, el problema del estado inicial. Reconocer que el sistema opera como una caché de último valor conocido transformó lo que parecía duplicación de código en dos estrategias deliberadas: suscripción continua y consulta puntual.
- El contraste entre el evento automático y el disparo manual nos llevó a preguntarnos por el costo real de obtener información: el sondeo consume batería aunque nada cambie, mientras la suscripción solo se activa ante un evento significativo. La eficiencia energética del software es, en suma, una cuestión de respeto.
- Estudiar `PendingIntent` nos permitió comprender la delegación de ejecución: el objeto no es un mensaje, sino un testigo de capacidad que autoriza al sistema a actuar en nombre de la aplicación. `FLAG_IMMUTABLE` y `RECEIVER_NOT_EXPORTED` materializan un modelo de seguridad que reduce deliberadamente la superficie de ataque.
- Desarrollar dos propuestas en paralelo resultó más fecundo que cualquiera por separado. Contrastar soluciones válidas con criterios distintos convirtió la revisión entre pares en un ejercicio de pensamiento crítico, y no en una simple validación mutua.

IV. RECOMENDACIONES

- Recomendamos adoptar una metodología de verificación más rigurosa, alineada con la investigación empírica: perfilado del consumo energético de cada estrategia de escucha, medición de la latencia entre la emisión del sistema y la recomposición, y comparación de resultados entre ambos desarrollos. No es posible afirmar una ventaja de rendimiento sin haberla medido, y ese hábito de cuantificar es lo que distingue una afirmación técnica de una simple conjetura.
- Recomendamos continuar la formación en los fundamentos formales que sustentan estas decisiones: los modelos de programación reactiva, el patrón observador y el esquema de permisos de Android. Describir con precisión física el almacenamiento de la batería, los límites térmicos de la carga y las limitaciones de los sensores del dispositivo resulta indispensable para no reproducir acriticamente patrones publicados que no se ajustan a las restricciones reales del hardware.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

- Recomendamos explorar una arquitectura híbrida que combine la suscripción a eventos con un modelo de estado propio, y que separe la lógica de negocio de la capa de presentación. Aislar el estado de la batería en un flujo de datos observable, al que la interfaz únicamente se suscriba, no solo mejora la testabilidad del sistema, sino que permite reutilizar esa lógica en un servicio en segundo plano o en pruebas automatizadas, donde no existe una composición que la sostenga.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.
- [2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.
- [3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.